

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 60%

QUESTIONS: 6

Total Marks:60

Annexure A

Application Based Questions

Code of Conduct:

I Sayed Fazal (192311291) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Application Based Questions

1. Use Case Diagram Analysis

A **Smart Hotel Reservation System** allows guests to search rooms, make reservations, cancel bookings, and process payments. Managers handle room allocation and reports, while payment services process transactions. The current design does not clearly distinguish actor responsibilities.

Analyze the system requirements, identify missing actors and use cases, and design an optimized Use Case Diagram showing proper relationships (include, extend, generalization) for better system clarity.

(10 Marks)

Answer:

(a) Identify the Actors

The actors involved in the Smart Hotel Reservation System are:

1. **Guest**
 - Searches rooms
 - Makes reservations
 - Cancels bookings
 - Makes payment
 - Views booking details
2. **Manager**
 - Allocates rooms
 - Manages room availability
 - Generates reports
3. **Payment Service (External System)**
 - Processes online payments
 - Confirms payment status

Missing Actor

4. **Administrator**
 - Manages users
 - Manages hotel information
 - Updates room details
 - Monitors system operations

(b) Identify Primary Use Cases

Guest

- Register/Login
- Search Rooms
- View Room Details
- Make Reservation
- Make Payment
- View Booking
- Cancel Booking

Manager

- Login
- Allocate Rooms
- Manage Room Availability
- Generate Reports

Administrator

- Login
- Manage Users
- Manage Room Information
- Update Room Details
- Monitor System

Payment Service

- Process Payment
- Verify Transaction

(c) Relationships

<<include>>

These are mandatory operations.

- **Make Reservation <<include>> Search Rooms**
- **Make Reservation <<include>> Check Room Availability**
- **Make Reservation <<include>> Make Payment**
- **Make Payment <<include>> Process Payment**
- **Generate Reports <<include>> Retrieve Booking Records**

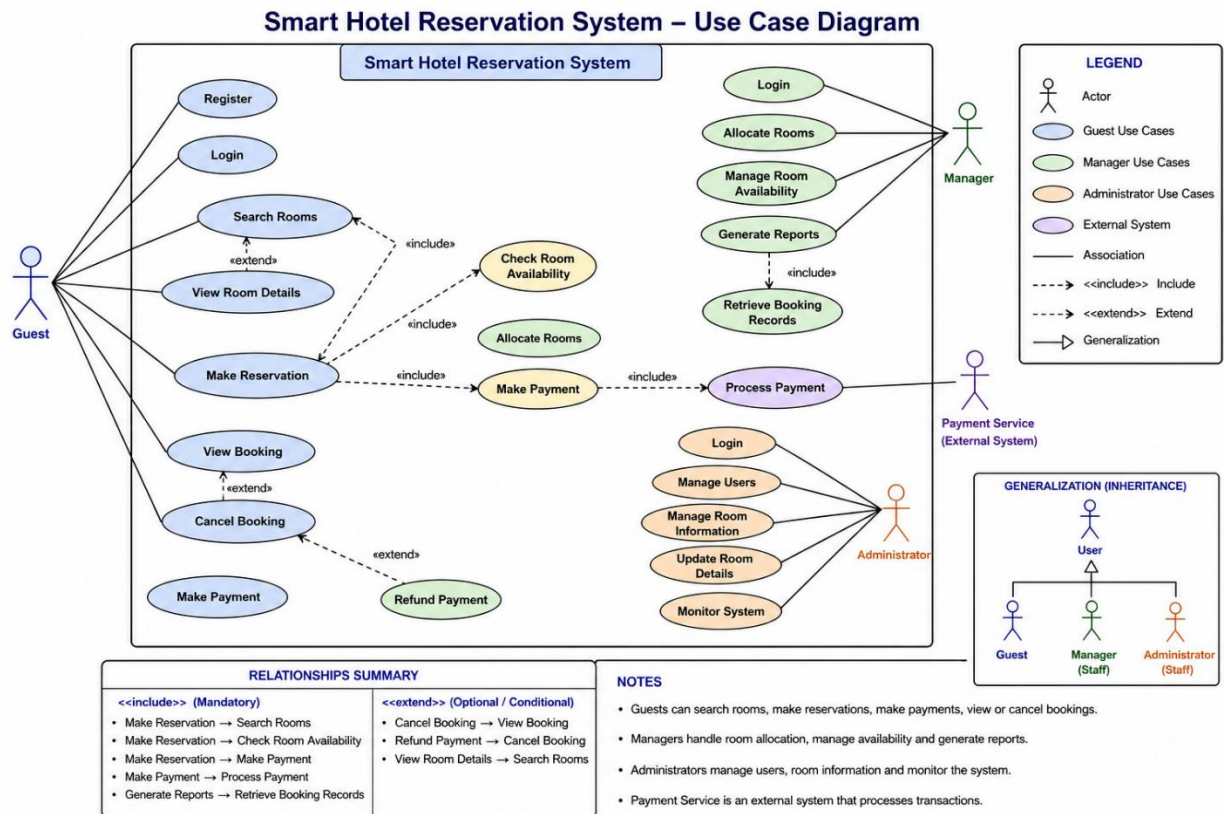
<<extend>>

These are optional or conditional operations.

- **Cancel Booking <<extend>> View Booking**
- **Refund Payment <<extend>> Cancel Booking**

- View Room Details <<extend>> Search Rooms

(d) Optimized Use Case Diagram



2. Class Diagram Design

A **University Management System** includes entities such as Student, Faculty, Course, Department, and Examination. The existing design lacks proper attributes, operations, and relationships among classes.

Analyze the system, identify relevant classes with attributes and methods, define relationships (association, aggregation, inheritance), and construct a well-structured Class Diagram.

(10 Marks)

(a) Identify Relevant Classes with Attributes and Methods

1. Student

Attributes

- studentID : String
- name : String
- email : String
- semester : int

Methods

- enrollCourse()
 - viewResult()
 - payFee()
-

2. Faculty

Attributes

- facultyID : String
- name : String
- designation : String
- specialization : String

Methods

- assignCourse()
 - conductExam()
 - uploadMarks()
-

3. Course

Attributes

- courseID : String
- courseName : String
- credits : int

Methods

- addStudent()
 - removeStudent()
 - updateCourse()
-

4. Department

Attributes

- departmentID : String
- departmentName : String
- location : String

Methods

- addCourse()
 - assignFaculty()
 - manageDepartment()
-

5. Examination

Attributes

- examID : String
- examDate : Date
- examType : String
- totalMarks : int

Methods

- scheduleExam()
- evaluateExam()
- publishResult()

(b) Define Relationships

Association

- Student enrolls in Course.
- Faculty teaches Course.
- Course has Examination.

- Student appears for Examination.

Aggregation

- Department contains multiple Courses.
- Department contains multiple Faculty members.

(c) Specify Multiplicity

Relationship	Multiplicity
Department → Course	1 : 1..*
Department → Faculty	1 : 1..*
Student → Course	0.. : 0..**
Course → Examination	1 : 1..*
Student → Examination	1 : 0..*
Faculty → Course	1 : 0..*

(d) Construct a Well-Structured Class Diagram

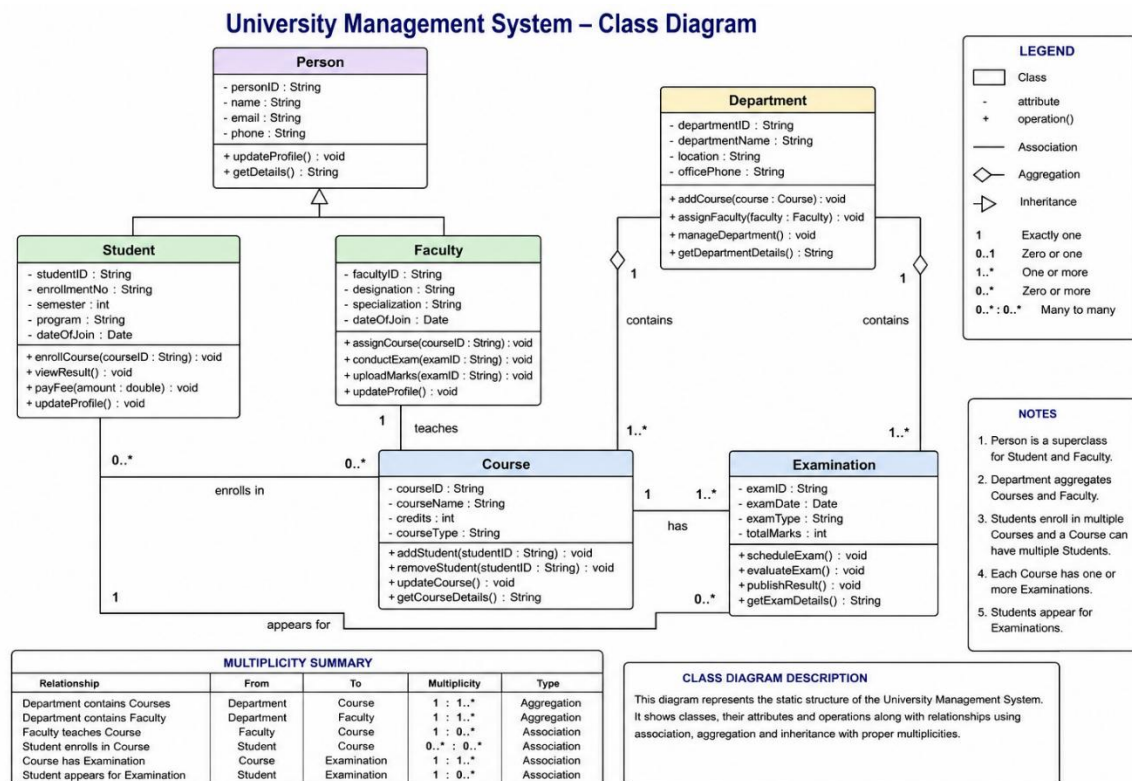


Diagram Components

Classes

- Person
- Student

- Faculty
- Department
- Course
- Examination

Inheritance

- Person $\rightarrow$ Student
- Person $\rightarrow$ Faculty

Associations

- Student $\leftrightarrow$ Course
- Faculty $\leftrightarrow$ Course
- Course $\leftrightarrow$ Examination
- Student $\leftrightarrow$ Examination

Aggregations

- Department $\diamond$ — Course
 - Department $\diamond$ — Faculty
-

3. Sequence Diagram Evaluation

In a **Vehicle Rental System**, a customer searches for a vehicle, books it, makes payment, and receives a rental confirmation. The interaction among system components is incomplete and poorly documented.

Analyze the process flow, identify missing objects and message sequences, and design a detailed Sequence Diagram representing proper interaction among system components.

(10 Marks)

(a) Analyze the Process Flow

The Vehicle Rental System follows this sequence of activities:

1. Customer searches for available vehicles.
2. System displays the available vehicles.
3. Customer selects a vehicle.
4. Customer submits the booking request.
5. System checks vehicle availability.
6. Vehicle is reserved.
7. Customer proceeds with payment.
8. Payment Gateway processes the payment.
9. Payment confirmation is returned.
10. System stores the booking information.
11. Confirmation notification is sent to the customer.

(b) Identify Missing Objects

The objects involved in the sequence are:

1. Customer
2. Vehicle Rental System
3. Vehicle Database
4. Payment Gateway
5. Booking Database
6. Notification Service

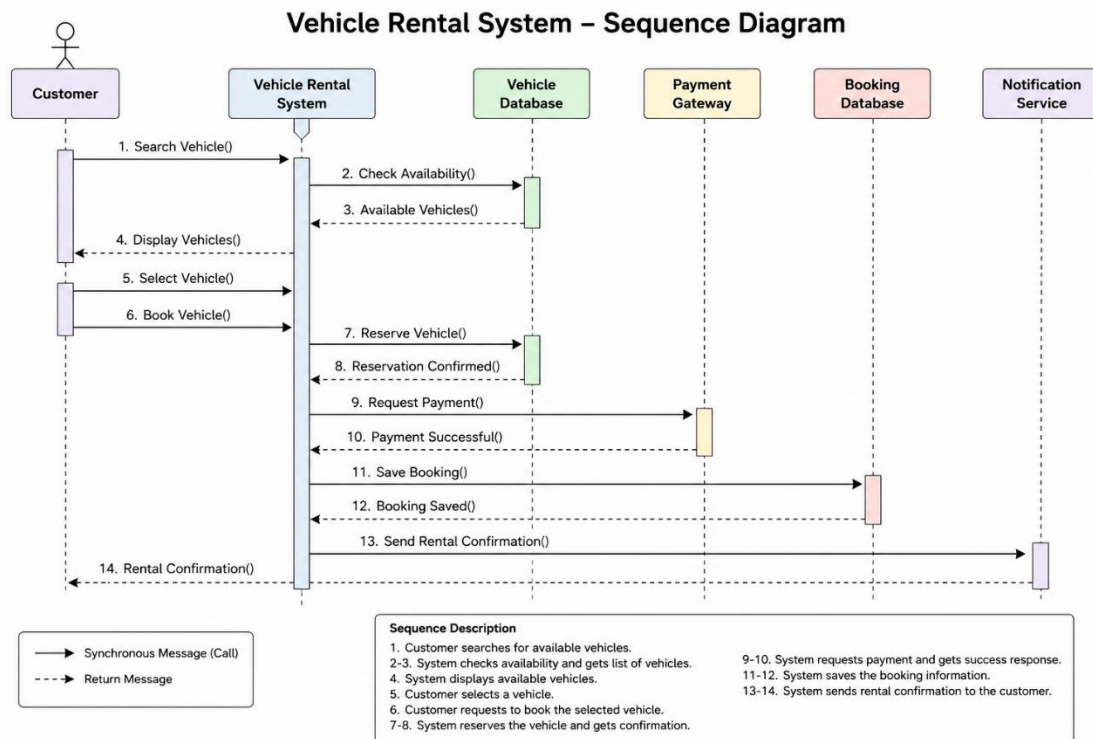
(c) Define Message Sequence

The interaction between the objects is as follows:

1. Customer → Vehicle Rental System : **Search Vehicle()**
2. Vehicle Rental System → Vehicle Database : **Check Availability()**
3. Vehicle Database → Vehicle Rental System : **Available Vehicles()**
4. Vehicle Rental System → Customer : **Display Vehicles()**
5. Customer → Vehicle Rental System : **Select Vehicle()**
6. Customer → Vehicle Rental System : **Book Vehicle()**
7. Vehicle Rental System → Vehicle Database : **Reserve Vehicle()**
8. Vehicle Database → Vehicle Rental System : **Reservation Confirmed()**

9. Vehicle Rental System → Payment Gateway : **Request Payment()**
10. Payment Gateway → Vehicle Rental System : **Payment Successful()**
11. Vehicle Rental System → Booking Database : **Save Booking()**
12. Booking Database → Vehicle Rental System : **Booking Saved()**
13. Vehicle Rental System → Notification Service : **Send Rental Confirmation()**
14. Notification Service → Customer : **Rental Confirmation()**

(d) Sequence Diagram Design



4. State Diagram Construction

A **Traffic Signal Control System** operates through states such as Red Signal, Green Signal, Yellow Signal, Emergency Mode, and Maintenance Mode. The transitions between states are not clearly defined.

Analyze the system behavior, identify valid states and transitions, and construct a State Transition Diagram ensuring all events and conditions are properly represented.

(10 Marks)

(a) Analyze the System Behavior

The Traffic Signal Control System controls vehicle movement by switching between different signal states. Under normal operation, the signal cycles through **Red**, **Green**, and **Yellow**. During emergencies, the system enters **Emergency Mode** to give priority to emergency vehicles. When maintenance is required, the system switches to **Maintenance Mode**, suspending normal traffic operations until maintenance is completed.

(b) Identify Valid States

The valid states of the Traffic Signal Control System are:

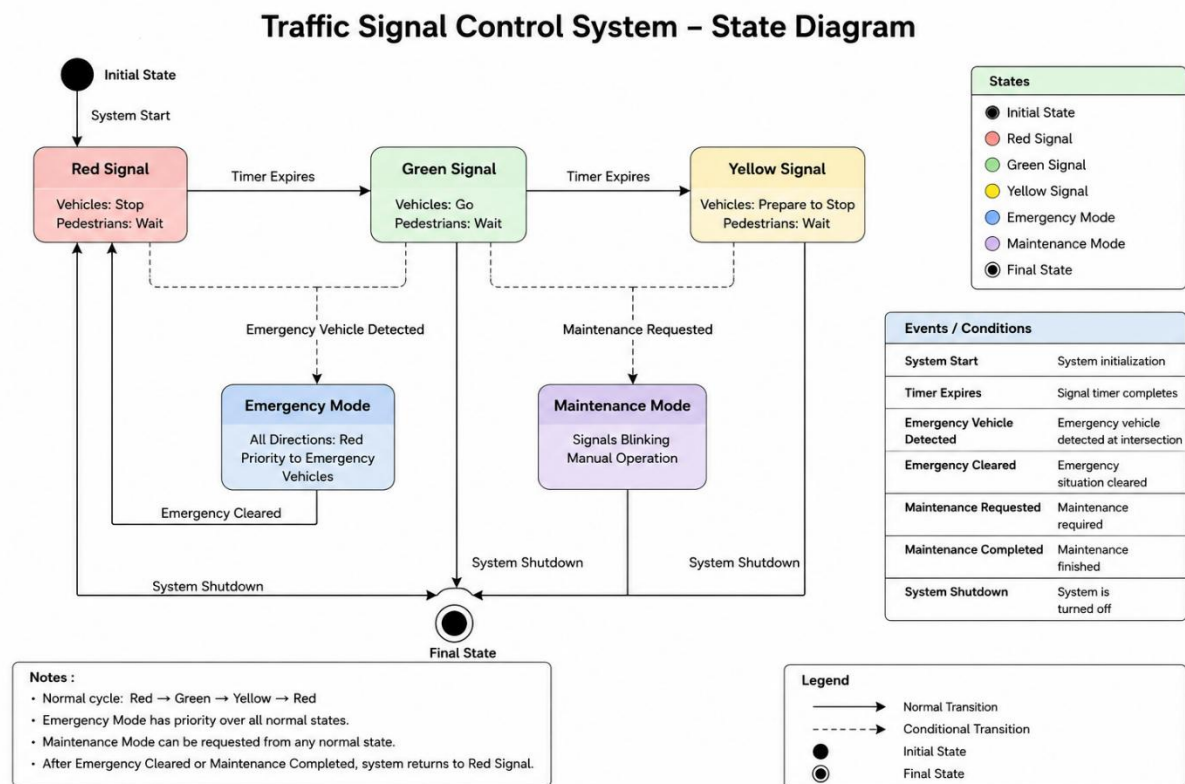
1. **Initial State**
2. **Red Signal**
3. **Green Signal**
4. **Yellow Signal**
5. **Emergency Mode**
6. **Maintenance Mode**
7. **Final State (System Shutdown)**

(c) Identify State Transitions and Events

Current State	Event/Condition	Next State
Initial State	System Start	Red Signal
Red Signal	Timer Expires	Green Signal
Green Signal	Timer Expires	Yellow Signal
Yellow Signal	Timer Expires	Red Signal
Red Signal	Emergency Vehicle Detected	Emergency Mode
Green Signal	Emergency Vehicle Detected	Emergency Mode
Yellow Signal	Emergency Vehicle Detected	Emergency Mode
Emergency Mode	Emergency Cleared	Red Signal
Red Signal	Maintenance Requested	Maintenance Mode

Current State	Event/Condition	Next State
Green Signal	Maintenance Requested	Maintenance Mode
Yellow Signal	Maintenance Requested	Maintenance Mode
Maintenance Mode	Maintenance Completed	Red Signal
Any State	System Shutdown	Final State

(d) State Transition Diagram Design



States Included

- Initial State
- Red Signal
- Green Signal
- Yellow Signal
- Emergency Mode
- Maintenance Mode
- Final State

5. Component Diagram Design

A **Smart Healthcare Portal** consists of modules such as Patient Management, Appointment Scheduling, Medical Records, Billing, and Notification Services. The current architecture does not clearly define component interactions and dependencies.

Analyze the system architecture, identify major components and their interactions, and design a Component Diagram showing dependencies and interfaces between modules.

(10 Marks)

(a) Analyze the System Architecture

The **Smart Healthcare Portal** is designed using a modular architecture where each component performs a specific function while interacting with other components through well-defined interfaces. The portal allows patients to manage their profiles, schedule appointments, access medical records, make payments, and receive notifications.

(b) Identify Major Components

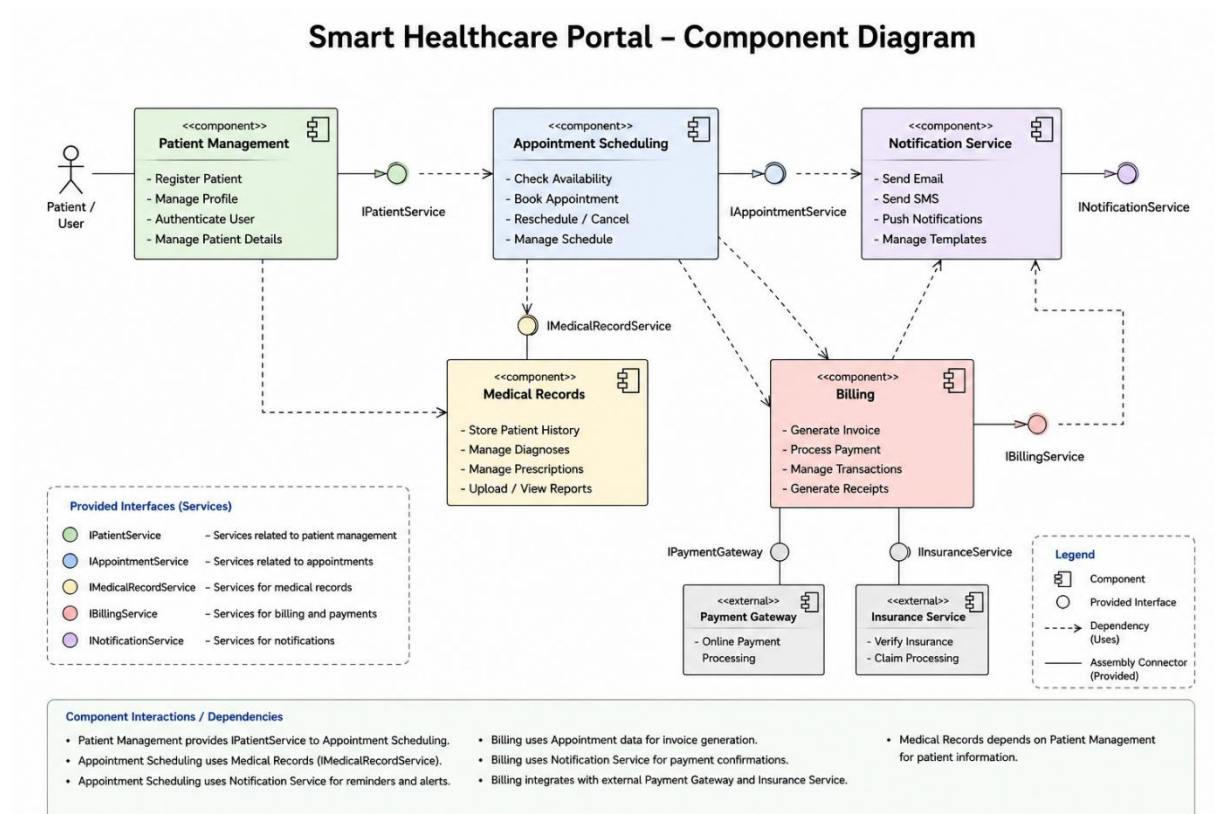
The major software components are:

1. **Patient Management**
 - Manages patient registration, login, and profile information.
2. **Appointment Scheduling**
 - Handles doctor availability, appointment booking, cancellation, and rescheduling.
3. **Medical Records**
 - Stores and retrieves patient medical history, prescriptions, reports, and diagnoses.
4. **Billing**
 - Generates invoices, processes payments, and maintains billing records.
5. **Notification Service**
 - Sends appointment reminders, payment confirmations, reports, and alerts via email or SMS.

(c) Identify Component Interactions and Dependencies

Component	Depends On	Purpose
Patient Management	Appointment Scheduling	Patient books appointments
Appointment Scheduling	Medical Records	Retrieves patient details and visit history
Appointment Scheduling	Notification Service	Sends appointment reminders
Medical Records	Patient Management	Accesses patient information
Billing	Appointment Scheduling	Generates bills after appointments
Billing	Notification Service	Sends payment confirmations
Notification Service	All Components	Delivers notifications and alerts

(d) Component Diagram Design



Components

- Patient Management
- Appointment Scheduling
- Medical Records
- Billing
- Notification Service

Interfaces

- **IPatientService**
- **IAppointmentService**
- **IMedicalRecordService**
- **IBillingService**
- **INotificationService**

Dependencies

- Patient Management → Appointment Scheduling
- Appointment Scheduling → Medical Records
- Appointment Scheduling → Notification Service
- Medical Records → Patient Management
- Billing → Appointment Scheduling
- Billing → Notification Service
- Notification Service provides notification services to all modules.

6. Deployment Diagram Design

A **Smart City Waste Management System** is deployed across IoT waste-bin sensors, edge devices, cloud servers, and monitoring dashboards. The current design lacks clarity regarding physical architecture and communication links.

Analyze the deployment environment, identify nodes, artifacts, and communication links, and construct a detailed Deployment Diagram representing the complete system architecture.

(10 Marks)

(a) Analyze the Deployment Environment

The **Smart City Waste Management System** is deployed across multiple physical devices and computing platforms. IoT sensors installed in waste bins collect fill-level and environmental data. The data is transmitted to nearby edge devices for preprocessing before being sent to cloud servers for storage, analytics, and route optimization. Municipal administrators access the processed information through a monitoring dashboard to monitor waste collection and make operational decisions.

(b) Identify Deployment Nodes and Artifacts

Deployment Nodes

1. IoT Waste-Bin Sensor Node
2. Edge Device (Gateway)
3. Cloud Server
4. Database Server
5. Monitoring Dashboard (Web/Mobile Client)

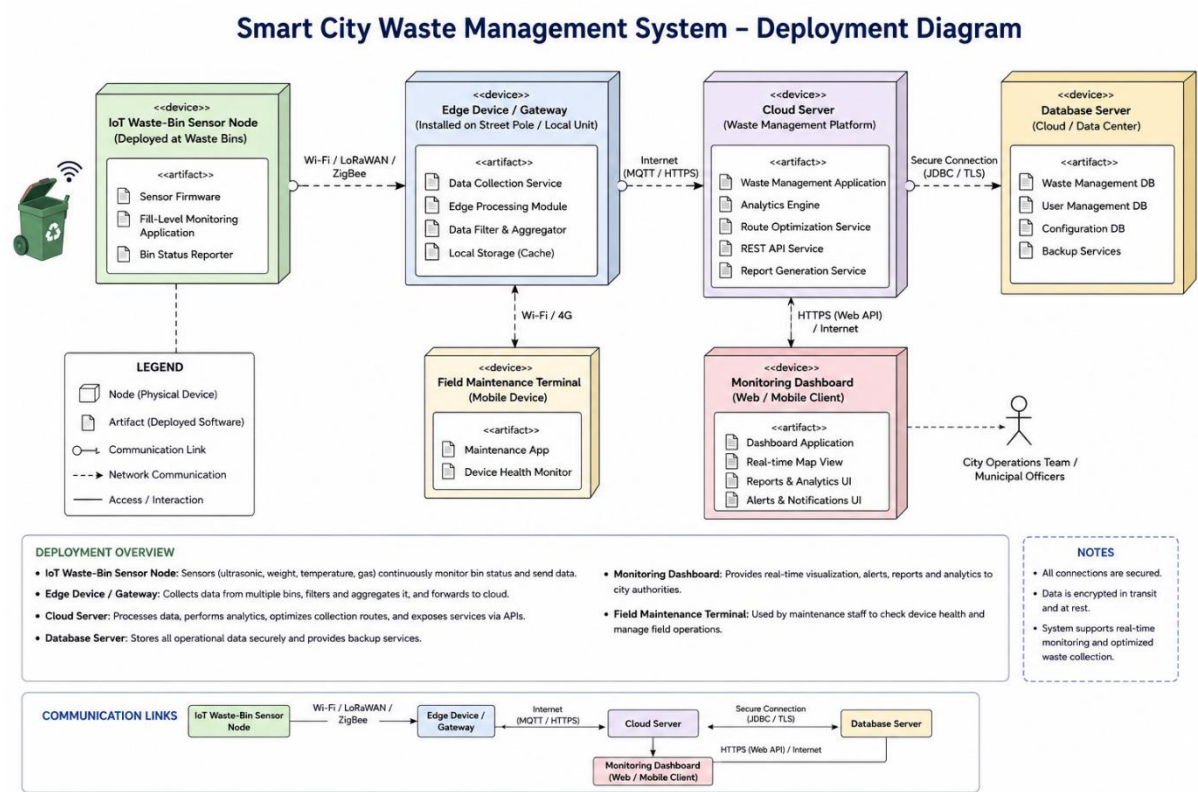
Artifacts Deployed

Node	Artifact
IoT Waste-Bin Sensor	Sensor Firmware, Fill-Level Monitoring Software
Edge Device	Data Collection Service, Edge Processing Module
Cloud Server	Waste Management Application, Analytics Engine, REST API
Database Server	Waste Management Database
Monitoring Dashboard	Dashboard Application (Web/Mobile UI)

(c) Identify Communication Links

Source Node	Destination Node	Communication
IoT Waste-Bin Sensor	Edge Device	Wi-Fi / LoRaWAN / ZigBee
Edge Device	Cloud Server	Internet (HTTP/HTTPS or MQTT)
Cloud Server	Database Server	Secure Database Connection
Monitoring Dashboard	Cloud Server	HTTPS/Web API
Cloud Server	Monitoring Dashboard	Real-time Status Updates and Reports

(d) Deployment Diagram Design



Nodes

- IoT Waste-Bin Sensor

- Edge Device
- Cloud Server
- Database Server
- Monitoring Dashboard

Artifacts

- Sensor Firmware
- Edge Processing Module
- Waste Management Application
- Analytics Engine
- Waste Management Database
- Dashboard Application

Deployment Relationships

- Sensor Firmware is deployed on the IoT Waste-Bin Sensor.
- Edge Processing Module is deployed on the Edge Device.
- Waste Management Application and Analytics Engine are deployed on the Cloud Server.
- Waste Management Database is deployed on the Database Server.
- Dashboard Application is deployed on the Monitoring Dashboard.

RUBRICS FOR CALCULATION

Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand